



Sistema de Reserva de Salas — IFPB

Avaliação dos Critérios Técnicos

Programação para Web 2 (PWEB2)

Resposta detalhada aos 7 tópicos avaliativos da disciplina, com evidências extraídas diretamente do código-fonte do projeto.

Projeto	SIRA v1.0.0 — release publicada
URL pública	https://gabemarques-intetsu.github.io/SIRA/
Stack	Vite ^8.0.10 + Vanilla JS (ESM) + LocalStorage + Node 24 LTS
Disciplinas	Programação para Web 2 + Engenharia de Requisitos de Software
Equipe	Gabriel Marques, Ian Lucas, Igor Gimenez, José Henrique, Pedro Sales
Total avaliado	10 pontos (7 tópicos)

Resumo dos critérios avaliados

Cada tópico abaixo aponta para a seção detalhada do documento. Nas seções individuais estão os trechos de código que comprovam o atendimento ao critério.

#	Critério	Peso	Atendido?
1	Programação Funcional (map, filter, reduce)	1,5 pts	Sim — extensivo
2	ESM — ECMAScript Modules (import/export)	1,0 pts	Sim — 19 arquivos JS modulares
3	Estruturação de Dados (arrays + LocalStorage)	2,0 pts	Sim — particionado por usuário
4	Manipulação do DOM + CRUD dinâmico	2,0 pts	Sim — 4 CRUDs completos
5	Tratamento de Eventos (click, input, keydown, popstate)	2,0 pts	Sim — 44+ listeners
6	Uso do Vite (Template Vanilla)	1,5 pts	Sim — Vite ^8.0.10
7	Uso de Bibliotecas (opcional)	—	Não usadas no runtime

1. Programação Funcional (1,5 pts)

Pontuação esperada: 1.5 / 1.5 pts

Pergunta: Como foram utilizados métodos de programação funcional, como `Array.map()`, `Array.filter()` e/ou `Array.reduce()`, para manipulação e transformação de dados dentro da aplicação?

Resposta

A aplicação foi projetada com **programação funcional explícita** como restrição pedagógica. Todo o módulo `src/utils/fp.js` é uma biblioteca de funções puras construída sobre `map`, `filter`, `reduce` e `some`, consumida por todos os módulos de página (calendário, reservas, salas, usuários, dashboard, aprovações, notificações, nova reserva).

Exemplo 1 — `Array.reduce` em `computeStats`

Em `src/utils/fp.js`, `reduce` agrega salas em um objeto de estatísticas consumido pelo Dashboard:

```
export const computeStats = (rooms, reservations, approvals) => { const roomStats = rooms.reduce( (acc, r) => { acc.total++; if (r.status === 'free') acc.free++; if (r.status === 'busy') acc.busy++; return acc; }, { total: 0, free: 0, busy: 0 }, ); const pending = reservations.filter((r) => r.status === 'pending').length; const approved = reservations.filter((r) => r.status === 'approved').length; // ... };
```

Exemplo 2 — `Array.filter` + `Array.some` em `filterByText`

Função pura genérica usada por todas as buscas textuais (reservas, salas, usuários):

```
export const filterByText = (list, query, fields) => { if (!query.trim()) return list; const q = query.toLowerCase(); return list.filter((item) => fields.some((f) => String(item[f] ?? '').toLowerCase().includes(q), ), ); };
```

Exemplo 3 — `Array.map` para construção de UI

Em `src/modules/calendar.js`, dois `map` aninhados constroem a grade semanal 7d x 12h sem nenhum loop imperativo:

```
const dayCols = DAYS.map((_, dayIdx) => el('div', { class: 'day-col' }, ...HOURS.map((_, hourIdx) => { const slot = el('div', { class: 'time-slot' }); // ... eventos do dia x hora return slot; })), ), );
```

Inventário de uso

Arquivo	O que demonstra	Trecho
<code>src/utils/fp.js</code>	<code>reduce</code> + <code>filter</code> + <code>some</code> — biblioteca funcional base	<code>computeStats</code> , <code>filterByText</code>
<code>src/modules/calendar.js</code>	<code>map</code> encadeado para grade semanal + <code>filter</code> de reservas do user	<code>DAYS.map(...HOURS.map(...))</code>
<code>src/modules/novaReserva.js</code>	<code>filter</code> de salas por tipo e por sobreposição de horário	<code>rooms.filter((r) => r.type === type)</code>

Arquivo	O que demonstra	Trecho
src/modules/notificat ions.js	map imutável para marcar 'lidas' sem mutar o array original	<code>notifications.map((n) => ({...n, read:true}))</code>
src/modules/reservati ons.js	filter + map encadeados em filterByText + filterByStatus	<code>filterByText > filterByStatus > map</code>
src/modules/approvals .js	filter para aprovações pendentes; map para construir cards	<code>approvals.filter().map((a) => card(a))</code>
src/modules/dashboard .js	consume computeStats (reduce) e renderiza 4 cards	<code>computeStats(rooms, res, app)</code>

2. ESM — ECMAScript Modules (1,0 pt)

Pontuação esperada: 1.0 / 1.0 pts

Pergunta: Como o código JavaScript foi organizado utilizando ESM (ECMAScript Modules), demonstrando a separação de responsabilidades entre arquivos e o uso de `import` e `export`?

Resposta

O projeto é totalmente ESM nativo (sem CommonJS, sem `require`). O `package.json` declara `"type": "module"` e o entry-point HTML carrega `<script type="module" src="/src/main.js">`. As responsabilidades estão separadas em **5 camadas**:

Camada	Pasta	Responsabilidade
Entrada	src/main.js	Bootstrap, roteador, telas de auth
Componentes	src/components/	Sidebar (US-06), Modal centralizado (US-25)
Módulos (páginas)	src/modules/	Dashboard, Calendário, Nova Reserva, Reservas, Aprovações, Salas, Usuários, Notificações
Dados	src/data/	store.js (CRUD + persistência), logins.json, seed.json
Utilitários	src/utlis/	dom.js (factory de elementos), fp.js (helpers funcionais)

Exemplo de import/export coordenado

O entry-point `src/main.js` importa páginas, componentes, utils e dados, e registra um *dispatcher* de rotas (função `→ renderer`):

```
// src/main.js
import { el } from './utlis/dom.js';
import { createSidebar } from './components/sidebar.js';
import { initModalListeners } from './components/modal.js';
import { renderDashboard } from './modules/dashboard.js';
import { renderCalendar } from './modules/calendar.js';
import { renderReservations } from './modules/reservations.js';
// ... mais 5 imports de páginas
import { tryRestoreSession, login, CURRENT_USER } from './data/store.js';
const PAGE_RENDERERS = {
  dashboard: renderDashboard,
  calendario: renderCalendar,
  // ...
};
```

Exports nomeados (sem default)

Todos os módulos exportam **símbolos nomeados** — facilita análise estática e tree-shaking do Vite. Exemplo de `src/data/store.js`:

```
export let CURRENT_USER = null;
export function loadCollection(collection, email = CURRENT_USER?.email) { ... }
export function saveCollection(collection, data, email = CURRENT_USER?.email) { ... }
export function login(email) { ... }
export const genId = generateId; // alias
export const saveRooms = saveRoom; // alias para compatibilidade
```

Métricas: 19 arquivos em `src/` (13 JS + 3 CSS + 2 JSON + 1 HTML), com importação cruzada apenas via paths relativos. Nenhum arquivo usa `module.exports` ou `require()`.

3. Estruturação de Dados — LocalStorage / Arrays (2,0 pts)

Pontuação esperada: 2.0 / 2.0 pts

Pergunta: Como os dados utilizados na aplicação foram organizados e manipulados, seja por meio de arrays em memória ou com a finalidade de persistência com LocalStorage?

Resposta

Toda a persistência usa **LocalStorage** **particionado por usuário**. A camada `src/data/store.js` abstrai o acesso e expõe getters/setters tipados por coleção. Em memória, todos os dados são manipulados como **arrays de objetos planos** (sem classes), o que casa naturalmente com `map/filter/reduce`.

Modelo de chaves no LocalStorage

Chave	Conteúdo	Quando é gravada
<code>sira-auth</code>	E-mail do usuário logado (string)	no <code>login()</code>
<code>sira-theme</code>	'dark' 'light'	no toggle de tema (US-09)
<code>sira:users</code>	Array de usuários globais (JSON)	<code>saveUsers()</code> — CRUD de usuários (US-22)
<code>sira:signups</code>	Array de solicitações de cadastro (JSON)	<code>renderSignup</code> — formulário público (US-04)
<code>sira_db//rooms.json</code>	Array de salas do usuário (JSON)	<code>saveRoom()</code> — CRUD de salas (US-21)
<code>sira_db//reservations.json</code>	Array de reservas do usuário	<code>saveReservation()</code> — Nova Reserva (US-15)
<code>sira_db//approvals.json</code>	Array de aprovações do usuário	<code>saveApproval()</code> — Fluxo US-11
<code>sira_db//notifications.json</code>	Array de notificações do usuário	<code>upsertApprovalNotification</code> — US-11

Funções genéricas de I/O

`loadCollection` e `saveCollection` em `store.js` são as únicas funções que tocam LocalStorage. Tudo mais é construído sobre elas:

```
// src/data/store.js
const DB_PREFIX = 'sira_db';
const buildCollectionKey = (email, collection) => `${DB_PREFIX}/${email}/${collection}.json`;
export function loadCollection(collection, email = CURRENT_USER?.email) {
  if (!email) return seedData[collection] ?? [];
  const raw = localStorage.getItem(buildCollectionKey(email, collection));
  if (!raw) return [];
  try { return JSON.parse(raw); } catch { return []; }
}
export function saveCollection(collection, data, email = CURRENT_USER?.email) {
  if (!email || !collection) return null;
  localStorage.setItem(buildCollectionKey(email, collection), JSON.stringify(data));
  return data;
}
```

Consolidação para admin

Quando o usuário logado é admin, getters concatenam dados de todos os usuários (via map e ...spread):

```
function consolidateCollection(collection) { const allUsers = getUsersGlobal();  
const consolidated = []; for (const user of allUsers) { const userData =  
loadCollection(collection, user.email); consolidated.push(...userData); } return  
consolidated; }
```

4. Manipulação do DOM e Componentes Dinâmicos (2,0 pts)

Pontuação esperada: 2.0 / 2.0 pts

Pergunta: Como o DOM foi manipulado para construção da interface? Descreva como os componentes foram criados dinamicamente de forma manual (sem uso de frameworks como Angular, React ou Vue), utilizando métodos como `createElement`, `appendChild` ou similares, a partir dos dados da aplicação. (Tente incluir no mínimo um CRUD)

Resposta

Nada de `innerHTML` com template strings: todo o DOM é construído via `document.createElement` encapsulado em um helper chamado `el()`, que aceita **tag, atributos e filhos variádicos**. Esse helper é a base de uma mini-API declarativa que substitui JSX/templates.

O helper `el()`

```
// src/utils/dom.js export function el(tag, attrs = {}, ...children) { const node = document.createElement(tag); for (const [k, v] of Object.entries(attrs)) { if (k.startsWith('on') && typeof v === 'function') { node.addEventListener(k.slice(2).toLowerCase(), v); } else if (k === 'style' && typeof v === 'object') { Object.assign(node.style, v); } else if (k === 'class') { node.className = v; } else { node.setAttribute(k, v); } } children.flat().forEach((c) => { if (c == null) return; node.appendChild(typeof c === 'string' || typeof c === 'number' ? document.createTextNode(String(c)) : c); }); return node; }
```

CRUDs implementados (todos via `el()` + `map`)

CRUD	Arquivo	Operações
Salas (US-21)	src/modules/rooms.js	Listar (map + grid), Criar/Editar (modal com inputs + checkboxes), Excluir (confirm)
Usuários (US-22)	src/modules/users.js	Tabela com map sobre <code>getUsers()</code> , Criar/Editar via modal, Remover com <code>confirm()</code>
Reservas (US-16/17/18)	src/modules/reservations.js	Listar com filter chips, Editar/Cancelar via modais, Exportar CSV via Blob
Nova Reserva (US-14/15)	src/modules/novaReserva.js	Form de busca, filter por overlap, <code>performReservation</code> persiste em <code>sira_db</code>

Trecho do CRUD de Salas (US-21)

O grid é construído com `filtered.map((r) => buildRoomCard(r, grid))`, e o modal de criar/editar reusa o helper `createModal`:


```
// src/modules/rooms.js – listagem
function refreshGrid(grid) { const all =
getRooms(); const filtered = filterRoomsByStatus( filterByText(all, searchQuery,
['name', 'block']), activeFilter, ); const cards = filtered.map((r) =>
buildRoomCard(r, grid)); const addCard = el('div', { class: 'room-card dashed',
onClick: () => openRoomModal(null, grid) }, el('span', { ... }, '+'), el('span', {
... }, 'Adicionar sala'), ); render(grid, ...cards, addCard); } // Exclusão imutável
+ persistência
function deleteRoom(id, grid) { confirm('Deseja excluir esta sala
permanentemente?', () => { saveRooms(getRooms().filter((r) => r.id !== id)); // ←
filter imutável refreshGrid(grid); toast('Sala removida.', 'success'); }); }
```

Observação: além do helper `el()`, há `btn()`, `badge()`, `tableRow()`, `toast()` e `confirm()` em `src/utils/dom.js` — todos chamando `createElement/appendChild` por baixo.

5. Tratamento de Eventos (2,0 pts)

Pontuação esperada: 2.0 / 2.0 pts

Pergunta: Como os eventos de interação do usuário (como cliques em botões, submissão de formulários ou mudanças em campos) foram tratados no JavaScript?

Resposta

Mais de **44 listeners** distribuídos pelos módulos cobrem todos os tipos de evento exigidos pela disciplina: click, input, keydown, change, popstate e MutationObserver. Os listeners são registrados de duas formas: via prop onClick/onInput no helper el() ou via addEventListener direto.

Exemplo 1 — click + keydown (Enter) na tela de login

```
// src/main.js – renderLogin el('input', { id: 'emailInput', class: 'auth-input',
type: 'email', onKeyDown: (ev) => { if (ev.key === 'Enter')
document.getElementById('loginBtn')?.click(); }, }, el('button', { id: 'loginBtn',
class: 'auth-btn auth-btn-primary', onClick: () => { const email =
document.getElementById('emailInput').value.trim().toLowerCase(); if (!email) {
alert('Informe um e-mail válido.');
```

Exemplo 2 — input para busca textual em tempo real

```
// src/modules/rooms.js const searchInput = el('input', { type: 'text', placeholder:
'Buscar sala...' }); searchInput.addEventListener('input', (e) => { searchQuery =
e.target.value; refreshGrid(grid); // re-renderiza a cada tecla });
```

Exemplo 3 — change em radio buttons (recorrência)

```
// src/modules/novaReserva.js recurSim.addEventListener('change', () =>
(weekDaysContainer.style.display = 'flex')); recurNao.addEventListener('change', ()
=> (weekDaysContainer.style.display = 'none'));
```

Exemplo 4 — popstate (botões voltar/avançar do navegador)

```
// src/main.js – bootstrap window.addEventListener('popstate', () => { let path =
window.location.pathname.replace(/^\/$/, ''); if (!path || !PAGE_RENDERERS[path])
path = 'calendario'; navigate(path); });
```

Exemplo 5 — MutationObserver para injetar UI mobile (US-08)

```
// src/main.js – drawer mobile const topbarObserver = new MutationObserver(() => {
const topbar = pageContainer.querySelector('.topbar'); if (topbar &&
!topbar.querySelector('.hamburger')) { topbar.prepend(el('button', { class:
'hamburger', onClick: openSidebar }, ...)); } });
topbarObserver.observe(pageContainer, { childList: true, subtree: true });
```

Observação: a delegação de eventos é feita também *globalmente* pelo initModalListeners() em src/components/modal.js — um único listener no document captura a tecla Escape e fecha qualquer modal aberto (T-25.2).

6. Uso do Vite (Template Vanilla) (1,5 pts)

Pontuação esperada: 1.5 / 1.5 pts

Pergunta: Como foi realizada a configuração e utilização do Vite com o template vanilla para criação e execução do projeto?

Resposta

O projeto usa Vite ^8.0.10 com o template **Vanilla** (sem React/Vue/Svelte). A configuração é mínima — só um vite.config.js com base condicional para GitHub Pages — porque o template vanilla já cobre tudo: HMR, tree-shaking, bundling de CSS, importação de JSON nativa.

vite.config.js

```
// vite.config.js import { defineConfig } from 'vite'; export default defineConfig({
  base: process.env.GITHUB_PAGES === 'true' ?
    `/${process.env.GITHUB_REPOSITORY.split('/')[1]}/` : '/', });
```

Em desenvolvimento (npm run dev) o base é /. Em build de produção para GitHub Pages, o workflow deploy.yml injeta GITHUB_PAGES=true e o base vira /SIRA/ (extraído de GITHUB_REPOSITORY).

Scripts npm

Script	O que faz
npm run dev	Inicia o dev server do Vite em http://localhost:5173 com HMR
npm run build	Compila e otimiza em dist/ (Rollup por baixo)
npm run preview	Serve dist/ localmente para testar o build de produção
npm run lint:check	Verifica ESLint sem corrigir (gate de CI)
npm run check-format	Verifica Prettier sem corrigir (gate de CI)

Build de produção (output real)

```
$ npm run build vite v8.0.10 building client environment for production... ✓ 23
modules transformed. dist/index.html 0.43 kB | gzip: 0.30 kB
dist/assets/index-CLysyH9Y.css 19.70 kB | gzip: 4.13 kB
dist/assets/index-OL1vGqFt.js 40.92 kB | gzip: 12.45 kB ✓ built in 96ms
```

Custo total da app: ≈16,9 KB gzipped (HTML + CSS + JS). Carrega em qualquer rede.

7. Uso de Bibliotecas (opcional)

Pergunta: Foi utilizada alguma biblioteca auxiliar (exceto frameworks de componentes como Angular, React ou Vue)? Se sim, explique qual foi utilizada e sua finalidade no projeto.

Resposta

Nenhuma biblioteca de runtime foi utilizada. O bloco dependencies do package.json está vazio — a aplicação final em produção carrega somente o JS escrito pela equipe.

```
// package.json { "name": "sira-sistema-de-reserva-salas-e-equipamentos", "version":  
"1.0.0", "type": "module", // ... "devDependencies": { "@eslint/js": "^9.39.4",  
"eslint": "^9.39.4", "eslint-config-prettier": "^10.1.8", "globals": "^17.6.0",  
"husky": "^9.1.7", "lint-staged": "^15.5.2", "prettier": "^3.8.3", "vite": "^8.0.10"  
} // sem "dependencies" }
```

Ferramentas de desenvolvimento (não vão pro bundle)

Os pacotes em devDependencies existem apenas em tempo de desenvolvimento (lint, format, hooks de commit, build). Nenhum deles é importado pelo código da aplicação:

- **Vite** — bundler e dev server (não vai para o bundle final).
- **ESLint + eslint-config-prettier + globals + @eslint/js** — linting estático.
- **Prettier** — formatação automática.
- **Husky + lint-staged** — hooks de pre-commit (rodam lint + format só nos arquivos alterados).

Justificativa pedagógica

A restrição da disciplina pedia **Vite com template Vanilla, sem React/Vue/Angular**. A equipe estendeu essa restrição também para bibliotecas utilitárias (Lodash, date-fns, uuid, RxJS, etc.) com dois objetivos:

- **Exercitar os primitivos do JavaScript:** o helper `el()` em `dom.js` é praticamente uma reimplementação minimalista de `React.createElement`; `fp.js` implementa do zero o que Lodash forneceria; IDs únicos vêm de `genId(prefix)` baseado em `Date.now()`.
- **Manter o bundle minúsculo:** 12,45 KB de JS gzipped é uma métrica que muitos projetos com dependência só atingem após code-splitting agressivo.

Resumo: esse item é opcional e o projeto opta por **não utilizá-lo**, documentando explicitamente a decisão.